General function.

These functions are used to handle NULL values in database.

NVL Function: substitutes an alternate value for NULL value.

Syntax: NVL(Arg1, replace_with)

Conditional Function: SQL provides conditional functions **DECODE** to impose conditions even in SQL statements.

DECODE Function: SQL equivalence of IF... THEN... ELSE conditional procedural statement. DECODE works with values/columns/expressions of all data types.

Syntax:

DECODE (expression, search, result [, search, result]... [, default])

Page No.
Date

SQL Functions

L supports inbuilt function (already implemented in SQL)

Syntax:

SELECT function (column_name) FROM table_name;

(i) Single Row function: functions that are one who work on single row and return one output per row.
eg: length and case conversion functions.

(ii) Multiple Row function: function that work upon group of rows and return one result for complete set of rows. Also called group function.

I) String functions.

(i) LCASE() or LOWER() : converts to lower case. OUTPUT
eg: SELECT LCASE ("INFORMATION"); → information

(ii) UCASE() or UPPER(): converts to uppercase
eg: SELECT UCASE ("Information"); → INFORMATION

(iii) CONCAT() : concatenation
eg: SELECT CONCAT ("I", "am", "Best"); → IamBest

(iv) SUBSTR() : returns characters from particular position.
eg: SUBSTR ("EASYCALCULATION", 5, 11); → CALCULATION
(1 2 3 4 5 → 11; 5 = position, 11 = length)

(v) INSTR(): gives position of first occurance of substring
eg: INSTR ("Footbarbar", "bar") → 5
(1 2 3 4 5)

(v) LENGTH() : length of substring in bytes including blank spaces

eg: LENGTH ("environment"); → 11
12345678910 11

(vi) TRIM() : returns string after removing all prefix or suffix spaces from given string

eg: TRIM (" India") → India

TRIM (" India ") → India

TRIM (LEADING "!" FROM "!!! India !!!");

output: India !!!

TRIM (TRAILING "!" FROM "!!! India !!!");

Output: !!! India

TRIM (BOTH "!" FROM "!!! India !!!");

Output: India

(vii) LPAD : gives string upto specified length with given character

eg: LPAD ('tech', 8, '0'); → 0000tech

II) Mathematical function.

(i) MOD() : returns remainder of one expression by dividing by another expression

eg: SELECT MOD (11, 4); → 3

(ii) ROUND : rounds up number upward or downward to nearest whole number.

eg: SELECT ROUND (4.65, 1); → 4.7

(iii) TRUNCATE: returns number as an argument truncated upto a specified number of decimal places.

eg: SELECT TRUNCATE (1.999, 1); → 1.9

Page No.
Date

III) Date / Time function.

(i) CURDATE() / CURRENT_DATE() : function returns current date in 'YYYY-MM-DD' or 'YYYYMMDD' format

eg: SELECT CURRENT_DATE(); → 10-May-2015 will be displayed as 2015-05-10

(ii) DATE() : function returns only DATE part from given date/time argument.

eg: DATE("2015-05-31 01:02:05"); → 2015-05-31

(iii) MONTH() : returns month part from date (1 to 12)

eg: SELECT MONTH("2015-05-31"); → 05

(iv) YEAR() : returns year part from date (range 1000 to 9999 or 0 for null date)

eg: SELECT YEAR("2015-05-31"); → 2015

(v) DAYNAME() : returns name of weekday from date given

eg: SELECT DAYNAME("2018-10-10"); → wednesday

(vi) DAYOFMONTH() : returns day of month for given date (1 to 31)

eg: SELECT DAYOFMONTH("2015-05-31"); → 31

(vii) DAYOFWEEK() : returns weekday number (1 for Sunday, 2 for monday & soon)

eg: SELECT DAYOFWEEK("2018-10-10"); → 4 bcz of wednesday

(viii) DAYOFYEAR() : returns day of year for given date (1 to 366)

eg: SELECT DAYOFYEAR("2007-07-09"); → 190

Page No.
Date

(ix) NOW() : returns current date and time in format 'YYYY-MM-DD HH:MM:SS' or 'YYYYMMDD HHMMSS'

eg: SELECT NOW(); → display today date.
2018-10-10 01:15:20

(x) SYSDATE() : returns current date & time at which function executes a value in 'YYYY-MM-DD HH:MM:SS' or 'YYYYMMDD HHMMSS'

eg: SELECT SYSDATE(); → Let current date is 10 October 2018 and time is 01:15:20 then, it displays 2018-10-10 01:15:20

(IV) Important SQL function.

(i) ADD-MONTHS : func^n adds specified number of months to a datetime value.

eg: SELECT ADD_MONTHS ('01- Aug -18', 3);
output → '01-NOV-18'

(ii) MONTHS_BETWEEN : func^n calculates no. of months b/w two dates

eg: SELECT MONTHS_BETWEEN ('6-Jul-05', '17-Jul-03'); output → 23.65

(iii) NEXT_DAY : returns date of first instance of particular day of week that follows the specified date.

eg: SELECT NEXT_DAY ('01-Aug-03', 'Tuesday');
Output → '05-Aug-03'

(iv) INITCAP : returns ~~capt~~ capitalise string/char (capitalise first letter of each word)

Page No.
Date

eg: SELECT INITCAP ('computer science');
output → Computer Science

(v) LAST_DAY : returns date of last day of month that contains date.
eg: LAST_DAY (2017-05-18) ⟶ 2017-05-31

DBMS

Database

A database is a seperate application that stores a collection of data. Each database has one or more distinct APIs for creating, accessing, managing, searching and replicating the data it holds

Nowdays, we use RDBMS to store and manage huge volume of data

RDMS: data is stored in different tables and relation are established using primary key or other keys known as foreign key.

Redundancy: storing data twice, redundantly to make the system faster

Primary key: A unique key. A key value cannot occur twice in one table and cannot be null.

Foreign key: is a linking pin b/w two tables

Compound key / composite key: is a key that consist of multiple columns, bcz one column is not sufficiently unique.

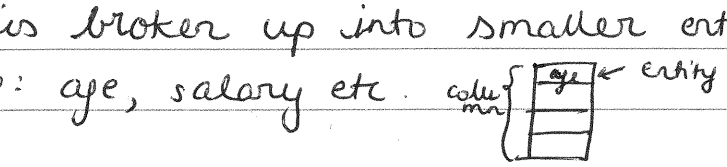
Index: resembles an index at back of book.

Referential Integrity: makes sure that foreign key value always point to an existing row.

default file size limit of a table is 4GB which can be increased is OS can handle it.

Table: table is a collection of related data enteries and it consist of rows & columns.

Field (column): Every table is broken up into smaller entities called field. eg: age, salary etc.



Record / row: each individual entry that exist in a table



Null value: blank / space.

SQL CONSTRAINT: constraint are rules enforced on data column on table. They are used to limit the the type of data that can go into a table. This ensures accuracy and reliability of data in database.

- Column level (applied to one column)
- table level (applied to whole table)

(i) NOT NULL CONSTRAINT: Ensures that a column cannot have null value

(ii) Default constraint: provides a default value for column when not specified

(iii) Unique constraint: ensures that all values are unique in column

(iv) Primary key: Uniquely identifies each row/record in database table / not null.

(v) Foreign key: " " a row) " " any another database table.

(vi) check constraint: ensures that all values in column satisfy certain condition

(vii) Index: use to create and retrieve data from database quickly.

(for unique key) if table is already there

```
ALTER TABLE CUSTOMER ADD PRIMARY KEY (ID);
```

example.

(to drop a default constraint)

→ ALTER TABLE CONSTRAINT

```
ALTER COLUMN SALARY DROP DEFUALT;
```

* for primary key →

```
ALTER TABLE CUSTOMER DROP PRIMARY KEY;
```

```
CREATE TABLE CUSTOMERS (
        ID INT                          NOT NULL ,
        NAME VARCHAR (20)               NOT NULL UNIQUE ,
        AGE INT                 NOT NULL CHECK (AGE >= 18),
        ADDRESS CHAR
        SALARY DECIMAL (18, 2) DEFAULT 5000.00 ,
        PRIMARY KEY (ID)
);                              or
                                PRIMARY KEY (ID, NAME)
```

If table is already created, then

```
ALTER TABLE CUSTOMERS
MODIFY SALARY DECIMAL (18,2) NOT NULL ;
"       "       "       "   " DEFAULT 5000.00 ;
"       AGE     INT     NOT NULL CHECK (AGE >= 18);
```

this line would change →

Index:

```
create index index_name
    ON table_name (col1, col2, ....)
```

ex:
```
create INDEX idx_age
    ON CUSTOMERS (AGE);
```

for drop:
```
ALTER TABLE CUSTOMERS
    DROP INDEX idx_age;
```

Foreign key

Let there be another table

```
CREATE TABLE ORDERS (
    ID INT NOT NULL,
    DATEN DATETIME,
    CUSTOMER_ID INT references CUSTOMER(ID),
    AMOUNT double,
    PRIMARY KEY (ID)
);
```

↑ foreigner key

if table already created

```
ALTER TABLE ORDERS
    ADD FOREIGN KEY (Customer_id) references
        customer(ID);
```

to drop

```
ALTER TABLE ORDERS
    DROP FOREIGN KEY;
```

Data Integrity

- **Entity Integrity**: there are no duplicate rows in a table
- **Domain Integrity**: Enforces valid entries for given column by restricting type, format, range
- **Referential Integrity**: Rows cannot be deleted, which are used by other record
- **User-defined Integrity**: Enforces some specific business rules that do not fall into entities, domain or referential integrity

Database Normalization:

is process of efficiently organizing data in database. There are two reason for normalization process.

(i) Eliminating redundant data, ex: storing the same data in more than one tables

(ii) Ensuring data dependies make sense.

- 1NF (first normal form)
- 2NF (second normal form)
- 3NF (third normal form)

INF :

- Define data items required, bcz they become the column in table.
- Ensure that there are no repeating groups of data
- Ensure that there is a primary key.

First rule: define data items and put in correct table.
ex: you put all columns relating to location of meeting in location table, member in memberdetails table etc.

Second rule: Ensure no repeating groups of data

ex:

ID	NAME	AGE	ADDRESS	ORDERS
100	Sachin	36	LSD	XL-100
100	Sachin	36	LSD	Tape
100	Sachin	36	LSD	Books

break in two table

CUSTOMER

ID	Name	age	address
100	Sachin	36	LSD

and relate with key

ORDER

ID	customer_id	orders
10	100	XL-100
11	100	Tape
12	100	Books

Third rule: Create a primary key for each table which have already created.

2NF: it should meet all rules of 1NF and there must be no partial dependies of any of column of primary key.

ex:
```
CREATE TABLE CUSTOMERS (
    cust_id INT NOT NULL,
    cust_name varchar(20) NOT NULL,
    order_id int " ,
    order_detail varchar(20) " ,
    sale_date datetime ,
    primary key (cust_id, order_id)
);
```

The table is in first norm.
but (cust_id) ~~and~~ cust_name is dependent on it
So, we need to break table in three parts

```
Create table orders (
    order_id not null,
    order_detail varchar(20) " ,
    primary key (order_id)
);
```

dependent so, no partial dependy

```
Create table customer (
    cust_id int not null,
    cust_name varchar(20) " ,
    primary key (cust_id)
);
```

dependent so, no partial dependy

```
create table customerorder (
    cust_id int not null,
    order_id int " ,
    sale_date datetime ,
    primary key (cust_id, order_id)
);
```

third table
sale_date is independent of customer and order
So, third table

Now there is no partial dependency.

condition for 3NF non-trivial functional dependencies

3NF : (i) It is in second normal form

(ii) all non primary keys are dependent on primary key

advantage is : (i) data duplicacy is ~~removed~~ reduced and data base becomes smaller

(ii) data integrity : all tables entities would be in same table

SQL is pronounced as "sequel".

MY SQL server — T-SQL
— ANSI SQL

SQL is followed by unique set of rules and guidelines called syntax.

SQL statement start with : select, insert, update, delete, alter, drop, create, use, show

ends with : semicolon (;)

SQL AND|OR

```
Select column1, column2, ....
From table_name.
Where cond^n-1 {AND|OR} cond^n-2
```

SQL IN clause :

```
where column_name IN
(val-1, val-2, ..., val-N);
```

SQL between

```
where column_name between val1 and val2;
```

SQL like

```
where column_name LIKE {Pattern};
```

SQL order by
where cond^n
Group by column_name

SQL Having
where cond^n
Group by column_name
having (arithmetic func^n cond^n)

SQL drop table
DROP TABLE table_name;

SQL create index
Create UNIQUE INDEX index_name
ON table_name (col^n1, col^n2, col^n3...);

SQL drop index.
ALTER TABLE table_name
DROP INDEX index_name;

SQL count
SELECT COUNT (column_name)
FROM table_name
WHERE condition;

Create table table_name(
col^n1 datatype,
col^n2 " ,
col^n3 " ,
:
col^n N " ,
Primary key (one or more col^n)
);

Rename table
ALTER TABLE table_name
RENAME TO new_table_name;

SQL Insert
Insert INTO table_name (col^n1, col^n2, ... col^n N)
VALUES (val1, val2, ... valN);

SQL Update
UPDATE table_name
SET col^n1 = val1, col^n2 = val2, ... col^n = valN
[WHERE COND^n];
May use or not

drop, use

SQL delete

```
Delete FROM table_name
WHERE {cond^};
```

SQL create, database

```
CREATE DATABASE database_name;
DROP     "       "      ;
USE      "       "      ;
```

SQL commit

```
COMMIT;
```

SQL Rollback.

```
Rollback;
```

OPERATORS IN SQL

arithmetic	comparision	Logical (to compare value to all values)	operator to negate cond^
-	= equal to	ALL	
+	!= same	AND	
*	<>	ANY	
/	> / >=	between	
% modulus	< / <=	Exists	
	!<	IN	
	!>	NOT	
		OR	
		IS NULL	
		UNIQUE	

Alias

```
Select count(*) AS "RECORDS" FROM CUSTOMERS;
```

Count total no of rows in table customers

```
Select current_timestamp;
```

gives date.

```
select getdate();
```

Output : 2009-11-12 06:40:23

SHOW databases; it will show all the existing databases present so that you can choose

USE khushbu; if khushbu is existing databasename.

ie USE databasename;

DESC → shows description
customers; table name

it will show see the., if table has been created or not succesfully.

Insert:

```
INSERT INTO CUSTOMERS (ID, NAME, AGE, ADDRESS, SALARY)
VALUES (1, 'Ramesh', 32, 'Ahmedabad', 2000.00);
```

OR

```
INSERT INTO CUSTOMERS
VALUES ( 7, 'Khushbu', 24, 'Gossip', 10000.00);
```

Like clause : is used to compare a value to similer values using wildcard operator.

(i) percent sign (%)

(ii) underscore (_)

SELECT FROM table_name

(i) WHERE column LIKE 'XXXX%'
WHERE salary LIKE '200%' find any value that start with 200

(ii) " " " '%200%' " " have 200 in any position

(iii) " " " '_00%' find any value that have 00 in second and third position

(iv) " " " '2=%_%' find any value that start with 2 and are at least 3 character in length.

(v) " " " '%2' Find any value that end with 2

(vi) " " " '_2%3' Find any value that have 2 in second position and ends with 3

(vii) " " " '2___3' Find any value in a five-digit number that start with 2 and end with 3

TOP clause : used to fetch a top N number or X percent records from table

SELECT TOP number/percent column_name(s)
FROM table_name
WHERE [cond^n]
↑ depending (may be used or not)

SELECT TOP 3 * from customers

MYSQL uses LIMIT instead of TOP → Select * from customer limit 3;

Oracle " ROWNUM " " " → Select * from customer where ROWNUM<= 3;

ORDER BY : used to sort data in ascending or descending order based on one or more column. (by default ascending order)

```
Select column_list
FROM table_name
[WHERE cond^n]
[ORDER BY col^n1, ---.. col^nN ] [ASC|DSC];
```

ex: select * from customers
order by Name, Salary ;

```
select * from customers
order by NAME desc;
```

GROUP BY: used in collaboration with SELECT statement to arrange identical data into groups.

```
SELECT col^n 1, col^n 2
FOM table_name
WHERE [cond^n]
Group by col1, col2
order by col1, col2
```

eg:

```
Select name, salary
from customers
group by name
```

DISTINCT keyword: eliminate all duplicate records and fetching only unique records

```
Select distinct col1, col2
from table_name
where cond^
```

eg:
```
Select distinct salary
from customers
order by salary;
```

→ Join clause: used to combine records from two or more tables. A Join is means for combining fields from two tables by using Values common to each.

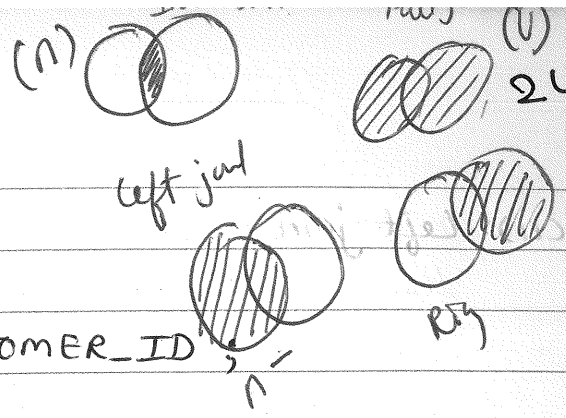
all example would be based on these tables

Customer table

ID	Name	age	address	salary
1	Ramesh	32	Ahmedabad	2000.00
2	Khilan	25	Delhi	1500.00
3	Kaushik	23	kota	2000.00
4	chaitali	25	mumbai	6500.00
5	Hardik	27	Bhopal	8500.00
6	Komal	22	MP	4500.00
7	muffy	24	Indore	10000.00

order table

OID	Date	customer_id	Amount
102	2009-10-08	3	3000
100	2009-10-08	3	1500
101	2009-11-20	2	1560
103	2008-05-20	4	2060




```
SELECT  ID, NAME, AGE, AMOUNT
FROM    CUSTOMERS, ORDERS
WHERE   CUSTOMER.ID = ORDER.CUSTOMER_ID;
```

output

ID	Name	age	amount
3	kaushik	23	3000
3	kaushik	23	1500
2	khilan	25	1560
4	chaitali	25	2060

several operators are used to join tables

=, <, >, <>, <=, >=, !=, BETWEEN, LIKE and NOT

SQL Join types.

(1) Inner Join: returns rows where there is match in both tables.

```
SELECT ID, NAME, AMOUNT, DATE
FROM CUSTOMERS
INNER JOIN ORDERS
ON CUSTOMER.ID = ORDERS.CUSTOMER_ID;
```

output:

ID	Name	Amount	date
3	kaushik	3000	2009-10-08
3	kaushik	1500	2009-10-08
2	khilan	1560	2009-11-20
4	chaitali	2060	2008-05-20

(ii) Left join: returns all rows from left table, even if there are no matches in right table

```
SELECT  ID, NAME, AMOUNT, DATE
FROM    CUSTOMERS
LEFT JOIN  ORDERS
ON  CUSTOMER.ID = ORDERS. CUSTOMER_ID;
```

output:

ID	Name	Amount	Date
1	Ramesh	NULL	NULL
2	khilar	1560	2009-11-20
3	kaushik	3000	" - 10 - 08
3	"	1500	2009 - 10 - 08
4	Chaitali	2060	2008 - 05 - 20
5	Hardik	NULL	NULL
6	komal	"	"
7	muffy	"	"

(iii) Right join: returns all rows from right table, even if there is no match in left table.

same as above: 3rd line would be: RIGHT JOIN ORDERS

ID	Name	Amount	Date
3	kaushik	3000	2009 - 10 - 08
3	kaushik	1500	" " 08
2	khilar	1560	2009 - 11 - 20
4	chaitali	2060	2008 - 05 - 20

MYSQL support UNION ALL

(iv) FULL JOIN: combines result of both left and right outer joins. (ie left and right joins would be done).

```
SELECT ID, NAME, AMOUNT, DATE
FROM CUSTOMERS
FULL JOIN ORDERS
ON CUSTOMER.ID = ORDERS.CUSTOMER_ID;
```

Output

ID	Name	Amount	date
1	Ramesh	NULL	NULL
2	khilan		
3	kaushik		
3	kaushik		
4	chaitali		
5	Hardik	NULL	NULL
6	komal	NULL	NULL
7	muffy	NULL	NULL
3	kaushik		
3	kaushik		
2	khilan		
4	Chaitali		

Would be filled accordingly

(v) SELF JOIN:

```
SELECT  a.ID, b.Name, a.salary
FROM    customers a, Customers b
WHERE   a.Salary < b.salary;
```

(vi) CARTESIAN JOIN OR CROSS JOIN: returns cartesian product of sets of records from two or more joined tables

```
SELECT  ID, NAME, AMOUNT, DATE
FROM    CUSTOMERS, ORDERS;
```

ID	NAME	AMOUNT	DATE
1	Ramesh	3000	2009-10-08
1	Ramesh	1500	"
1	"	1560	2009-11-20
1	"	2060	2008-05-20
2	Khilan	3000	2009-10-08
2	"	1500	"
2	"	1560	2009-11-20
2	"	2060	2008-05-20
⋮			
	and same goes on for all 7.		
⋮			
7	Muffy	3000	"
7	"	1500	"
7	"	1560	"
7	"	2060	"